



BOOTCAMP FRONTEND & REACT · SEMANA 3 DE 8

JavaScript

Clase 7 — La tienda empieza a pensar · variables, tipos, operadores, funciones y métodos de arrays

L · M · V

Proyecto TechCart

STEERCLAVE

VIENEN DE JAVA: LA LÓGICA YA LA TIENEN, HOY CAMBIA LA SINTAXIS

Agenda

1. Qué es JavaScript y dónde vive

los tres lenguajes · el script con defer · la consola

2. Variables y tipos de datos

const / let · primitivos · typeof · tipado dinámico

3. Operadores

=== y == · valores falsos · ternario · ?? y ?.

4. Estructuras de control

if · switch · for · for...of · while

5. Funciones

declaración, expresión y flecha · valor por defecto

6. Arrays y objetos

el catálogo de TechCart convertido en datos

7. map · filter · reduce

y los acompañantes: find, some, every, includes, sort

8. Cierre

errores frecuentes, resumen, 5 tareas y anticipo



Receso de 15 min después de las funciones · hoy la salida va a la **consola del navegador**

Dos de memoria

¿Qué me da declarar `--color-marca` dentro de `@theme`?

¿Qué significa el prefijo `md:` en una utilidad?

LO DIJIMOS EN LA CLASE 1 · HOY LLEGAMOS AL TERCERO

Los tres lenguajes de la web

HTML

Estructura. Qué hay en la página:
un título, una lista, un formulario.

CSS

Presentación. Cómo se ve eso que
hay: color, tamaño, posición.

JavaScript

Comportamiento. Qué hace la
página: calcular, decidir,
responder.

EL NAVEGADOR LEE EL HTML DE ARRIBA ABAJO · DÓNDE SE PONE EL SCRIPT IMPORTA

Dónde vive el `<script>`

en el `<head>`, pelado

El navegador se detiene, descarga y ejecuta antes de seguir dibujando. Página en blanco mientras tanto.

al final del `<body>`

Cuando el navegador llega, ya dibujó todo. Funciona, y es lo que van a ver en tutoriales.

defer en el `<head>`

Descarga en paralelo y ejecuta cuando la página está lista. **La que elegimos.**

```
<script defer src="./js/main.js"></script>
```

F12 → PESTAÑA CONSOLE · ES DONDE SE TRABAJA, NO UN RINCÓN DE CURIOSOS

La consola es la herramienta de hoy

console.logimprime · es el **System.out.println** de ustedes**console.table**dibuja un arreglo de objetos como **tabla****console.warn**

avisa, en amarillo

console.error

marca un problema, en rojo

y también se escribe ahí directo

```
> 2 + 2  
< 4
```

Un laboratorio: se prueba una idea sin recargar y sin tocar ningún archivo.

EN JS NO SE DECLARA EL TIPO: LO PONE EL VALOR

Guardar datos: **const** y **let**

const

no se puede reasignar · es el **final** de Java · **úsala por defecto**

let

variable que sí puede cambiar de valor

var

la forma vieja · alcance de función · no se usa

JAVA

```
int precio = 1999;  
final double IGV = 0.18;
```



JAVASCRIPT

```
let precio = 1999;  
const IGV = 0.18;
```

typeof dice el tipo · EN JS HAY UN SOLO TIPO NÚMERO, NO INT NI DOUBLE

Los tipos de datos

string

texto, entre comillas

number

enteros y decimales, juntos

boolean

true / false

undefined

existe, pero sin valor todavía

null

vacío, y es a propósito

symbol · bigint

existen; no los usamos en el curso

Java es de tipado **estático**. JavaScript es de tipado **dinámico**: la variable no tiene tipo, lo tiene el valor.

```
let dato = 5 // number
dato = "cinco" // string, y JS lo
permite
```

Cómodo al escribir, caro cuando el programa crece. La solución tiene nombre: **TypeScript**.

LAS CUATRO FAMILIAS · IDÉNTICAS A JAVA, SALVO UNA

Operadores

`+` `-` `*` `/` `%` `**`

aritméticos · el `%` es el resto · el `+` además junta textos

`=` `+=` `-=` `*=` `/=`

de asignación · a `+= 1` suma y guarda en un paso

`>` `<` `>=` `<=` `===` `!==`

de comparación · devuelven true o false

`&&` `||` `!`

lógicos · y · o · no

```
const precioMacBook = 1999.99
precioMacBook * (1 + IGV) // 2359.9882 → el precio con IGV, con todos sus decimales
7 % 2 // 1 → el resto de la división
```

EL ERROR NÚMERO UNO DE QUIEN EMPIEZA EN JAVASCRIPT

El gran tema: `===` frente a `==`

`===`

`"5" === 5 → false`

Compara **valor Y tipo**. Es lo correcto. **Úsalo siempre**,
y `!==` para distinto.

`==`

`"5" == 5 → true`

Convierte los tipos **a escondidas** antes de comparar.
Causa errores que nadie ve.

Valores que son **falsos por naturaleza**: `0` · `""` · `null` · `undefined` · `NaN` · `false`. Todo lo demás es verdadero.

UNO QUE YA CONOCEN DE JAVA Y DOS QUE EN JAVA NO EXISTEN

Ternario, ?? y ?.

? :

el ternario: un if/else que elige entre **dos valores** `precio > 1000 ? "premium" : "accesible"`

??

coalescencia nula: usa lo de la derecha **solo si** lo de la izquierda es null o undefined

?.

encadenamiento opcional: lee la propiedad **sin reventar** si el objeto no existe

```
const descuento = 0
descuento ?? "sin descuento" // 0 → ?? deja pasar el cero
descuento || "sin descuento" // "sin descuento" → || se lo come
```

CASI IDÉNTICO A JAVA · ESTO VA RÁPIDO

Estructuras de control

if / else

la decisión de siempre · mismas llaves, mismos paréntesis

switchuna variable contra **varios valores fijos** · con case, default y break (salvo que cada rama devuelva)**for**el clásico, con contador · **cuando hace falta el índice****for...of**recorre los elementos · es el **for-each** de Java, con of en vez de dos puntos**while**

cuando no se sabe cuántas vueltas son, solo cuándo parar

LA MISMA FUNCIÓN ESCRITA DE TRES MANERAS · HAY QUE SABER LEER LAS TRES

Funciones y la función flecha

la misma función, tres escrituras

```
function conIgv(p) { return p * 1.18 } // declaración
const conIgv = function (p) { return p * 1.18 } // expresión
const conIgv = (p) => p * 1.18 // flecha
```

las tres formas de la flecha

```
(p, pct = 10) => { return p - p * pct / 100 } // con llaves → return obligatorio
(p) => p * 1.18 // una expresión → return implícito
p => "$" + p.toFixed(2) // un solo parámetro → sin paréntesis
```

 **Java:** la flecha es la **lambda**. Ustedes escriben `p -> p * 1.18`; aquí la flecha lleva dos rayitas.

LA IDEA QUE SOSTIENE TODO LO QUE VIENE DESPUÉS DEL RECESO

Una función es un **valor**

```
function aplicar(precio, transformacion) {  
  return transformacion(precio) // la función es el ARGUMENTO  
}  
  
aplicar(1000, conDescuento) // 900  
aplicar(1000, conIgvFlecha) // 1180
```

Si una función se puede **pasar** a otra función, entonces un arreglo puede recibir una función que diga qué hacer con cada elemento. Eso es **map**, **filter** y **reduce**.

LISTA ORDENADA · EL ÍNDICE EMPIEZA EN 0, ASÍ QUE EL ÚLTIMO ES LENGTH - 1

Arrays

```
const categorias = ["laptops", "audio"]  
categorias[0] // "laptops"  
categorias.length // 2
```

 **Java:** su array o su ArrayList, pero sin declarar el tipo ni el tamaño. Crece solo.

push

agrega **al final**

pop

saca **el último**

unshift

agrega **al inicio**

shift

saca **el primero**

splice

saca o inserta **desde el medio**

Objetos

```
const producto = {  
  nombre: "iPhone 13 Pro",  
  precio: 1099.99,  
  categoria: "smartphones"  
}
```

dos formas de leer una propiedad

```
producto.nombre // con punto  
producto["nombre"] // con corchetes  
const k = "precio"  
producto[k] // ← esto el punto no lo  
puede hacer
```

 **Java:** un POJO con sus campos, o un HashMap. Aquí no hay que declarar la clase antes.

UN ARREGLO DE OBJETOS: LA FORMA NATURAL DE UN CATÁLOGO O UN CARRITO

El catálogo de TechCart, como **datos**

```
const productos = [  
  { id: 1, nombre: "MacBook Pro 14", precio: 1999.99, categoria: "laptops", stock: 5, destacado: true },  
  { id: 2, nombre: "iPhone 13 Pro", precio: 1099.99, categoria: "smartphones", stock: 8, destacado: false },  
  { id: 3, nombre: "iPad Mini 2021", precio: 499.99, categoria: "tablets", stock: 0, destacado: false },  
  { id: 4, nombre: "AirPods Max", precio: 549.99, categoria: "audio", stock: 3, destacado: false },  
]
```

Los cuatro de la página, más dos que agregamos con push. **Ojo:** productos es const y aun así se le agregan elementos: **const congela la variable, no el contenido.**

🔗 JAVA: ES LA API DE STREAMS, SIN EL `.STREAM()` NI EL `.COLLECT()`

El corazón del día: **map** · **filter** · **reduce**

map

transforma cada elemento →
arreglo del **MISMO** tamaño

filter

selecciona los que cumplen →
arreglo **MÁS CHICO** o igual

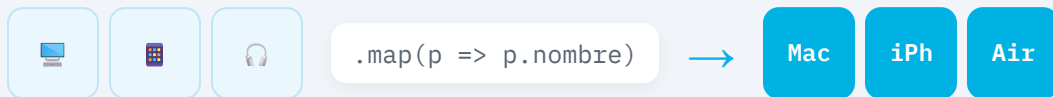
reduce

combina todo →
UN SOLO valor

Las tres reciben una **función flecha** y **ninguna toca el arreglo original**: las tres devuelven algo nuevo.

ENTRA UN ARREGLO, SALE UN ARREGLO DEL MISMO TAMAÑO, TRANSFORMADO

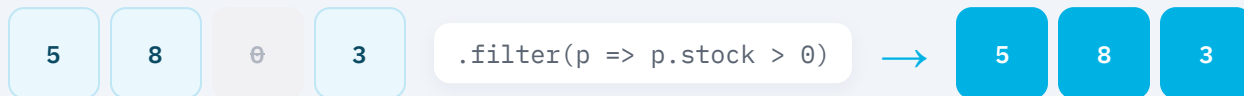
map — transformar



```
const nombres = productos.map(p => p.nombre)
const precios = productos.map(p => formatearPrecio(p.precio))
```

LA FLECHA DEVUELVE TRUE O FALSE: ES UNA PREGUNTA

filter — seleccionar



```
const enStock = productos.filter(p => p.stock > 0)
const laptops = productos.filter(p => p.categoria === "laptops")
productos.filter(p => p.precio < 600).map(p => p.nombre) // encadenados
```

UN ACUMULADOR QUE RECORRE EL ARREGLO Y TERMINA EN UN SOLO VALOR

reduce — combinar



```
const total = productos.reduce((suma, p) => suma + p.precio, 0)
// la flecha recibe DOS cosas, (acumulador, elemento), y el 0 del final es el VALOR INICIAL
```

Si el total sale raro o sale un objeto, el sospechoso número uno es el **valor inicial** que falta.

LOS VAN A USAR CASI TANTO COMO LOS TRES GRANDES

Los acompañantes

find devuelve **el primer elemento** que cumple, no un arreglo · si no hay, undefined

some ¿hay **al menos uno** que cumpla? · true o false

every ¿lo cumplen **todos**? · true o false

includes ¿está este valor en el arreglo? · también funciona sobre textos: ahí está el **buscador**

sort ordena · **Sí muta el original** y de fábrica ordena como **texto**: `[10,9,100].sort()` da `[10,100,9]`

```
productos.slice().sort((a, b) => a.precio - b.precio) // copia primero, números después
```

Una aclaración, para que nadie se vaya con la duda

Hoy la salida va a la **consola**

La página se ve **exactamente igual** que la clase pasada, y el HTML cambió en **una sola línea**: el `<script>`. Todo lo que calculamos hoy vive en la consola, y el visitante no lo ve.

Eso es a propósito. Escribir en la página desde JavaScript se hace con el **DOM**, y el DOM es un tema completo: la **Clase 9**. Hoy pusimos el cerebro; en dos clases le conectamos las manos, y el catálogo se va a pintar solo desde este mismo arreglo.

NO LOS VAMOS A CODEAR: SE LEEN PARA RECONOCERLOS CUANDO PASEN

Errores frecuentes

- 1 Usar == en vez de ===.** "5" == 5 da true y rompe una validación sin avisar.
- 2 Reasignar una const:** Uncaught TypeError: Assignment to constant variable. Y al revés: const **no** congela el contenido de un arreglo ni de un objeto.
- 3 Olvidar el return en una flecha con llaves.** Sin error y sin línea roja: el map devuelve un arreglo lleno de undefined.
- 4 Confundir map con forEach.** forEach no devuelve nada: la variable queda en undefined.
- 5 sort:** muta el original (copiar con slice()) y de fábrica ordena como texto (pasarle (a,b) => a-b).
- 6 El índice empieza en 0** y el último es length - 1. Pedir de más da Uncaught TypeError: Cannot read properties of undefined.
- 7 NaN al operar con texto.** "diez" * 2 da NaN, y NaN contagia toda la cuenta. Ojo: "10" + 2 da "102".
- 8 Olvidar el valor inicial de reduce.** Ese 0 del final es el que falta cuando el total sale mal.

LO QUE VIMOS HOY · Y CUÁNTO DE ESTO YA SABÍAN DE JAVA

Resumen

<script defer>

una línea, en el head

console.log / table

y warn y error

const · letalcance de bloque · var
nunca**typeof · primitivos**

tipado dinámico, no estático

=== siempre

y los valores falsos

?: ?? ?.

ternario y los dos modernos

**if · switch ·
bucles**

for, for...of y while

función flechatres formas · valor por
defecto**funciones = valores**

se pasan a otras funciones

arrays y objetos

el catálogo como datos

**map · filter ·
reduce**

no mutan · = streams

find · some · every

includes y sort

PARA CASA, CON LA GRABACIÓN AL LADO · TODAS SOBRE EL ARREGLO PRODUCTOS

5 tareas

FÁCIL

Replicar la clase y agregar un séptimo producto: comprobar `length`, la tabla y el valor del catálogo.

INTERMEDIA
1

El catálogo presentable: con `map`, nombre, categoría y precio formateado. Después, solo los que tienen stock.

INTERMEDIA
2

cuantosHayDe(categoría): función flecha con `filter` y `length`. "laptops" da 2, "televisores" da 0.

DIFÍCIL 1

resumenCarrito(items): devuelve { cantidad, total, envio }, con el envío gratis desde \$50.

DIFÍCIL 2

El más barato con stock, de dos maneras: `filter` + `sort` sobre una copia, y `filter` + `reduce`. ¿Cuál es más clara?

Próxima clase

JavaScript moderno

La forma corta de escribir lo que hoy escribimos largo: **desestructuración** para sacar varias propiedades en una línea, **template literals** para armar textos sin pegar pedazos con el más, **spread** para copiar y juntar, y **módulos** para partir el código en archivos que se importan, como los `import` de Java.

Y la pregunta que queda abierta: ¿cómo hacemos que esto salga en la página y no en la consola? Eso es el DOM, en la Clase 9.

Pista: en la consola, escriban
`document.querySelector("h1")` y miren lo que devuelve.